Sistemas Digitales — Semana 4

Interrupciones externas

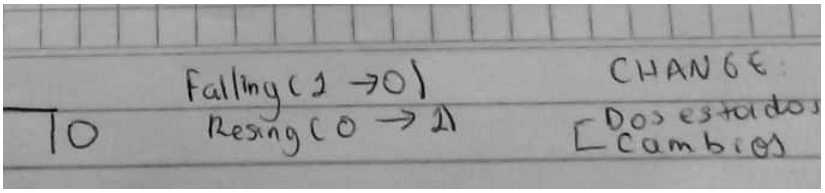
En los apuntes aparece sondeo y después interrupción. El sondeo es cuando el programa revisa a cada rato un pin, por ejemplo con digitalRead() dentro de loop(). Si el programa está ocupado, puede que no alcance a detectar una pulsación corta.

La interrupción es un aviso del hardware cuando ocurre un evento. En este caso el periférico GPIO detecta el cambio del pin y la CPU ejecuta una función especial llamada ISR. Luego continúa el programa. Es diferente a preguntar todo el tiempo si el botón cambió.

Flancos del botón

Con INPUT_PULLUP, el botón suelto se lee como 1 porque el pin está conectado a 3,3 V mediante la resistencia. Al presionar y unirlo a GND se lee como 0.

- FALLING: pasa de 1 a 0. Sirve para detectar cuando se presiona.
- RISING: pasa de 0 a 1. Detecta cuando se suelta.
- CHANGE: detecta ambos cambios, al presionar y al soltar.



attachInterrupt e ISR

Para activar una interrupción se usa attachInterrupt(). Allí indicamos qué pin vigilar, qué función se ejecutará y el tipo de flanco. Normalmente se configura en setup() una sola vez.

Ejemplo: attachInterrupt(digitalPinToInterrupt(21), isrBoton, FALLING);

La ISR (Interrupt Service Routine) es la función que atiende el evento. El código no la llama directamente: la ejecuta el hardware cuando se produce la interrupción.

Cuidado con los pines

En el ESP32 se debe revisar qué GPIO se utiliza. Los pines 34 a 39 son solo de entrada y no tienen pull-up interna, por lo que para un botón puede hacer falta una resistencia externa. También se debe cuidar los pines de arranque (strapping).

Sistemas Digitales — Semana 4

La ISR debe ser corta

Algo anotado en clase: “ISR debe ser corto”. La interrupción no debe quedarse haciendo trabajos largos porque puede retrasar otras tareas y provocar problemas como el watchdog. Mejor dejar una señal y procesarla después en loop().

- Evitar dentro de una ISR: delay(), impresiones por Serial y operaciones que tarden mucho.
- IRAM_ATTR: atributo que permite colocar el código de una ISR en RAM de instrucciones cuando se requiere que sea accesible incluso si la caché de Flash está deshabilitada.

Flash, IRAM y DRAM

La Flash guarda el programa aunque se apague el ESP32. La SRAM es volátil; dentro del microcontrolador se emplean zonas para instrucciones (IRAM) y datos (DRAM). La memoria caché ayuda a acceder al código que está en Flash.

Variable volatile y bandera

Si una variable cambia dentro de la ISR y también es consultada por loop(), podemos declararla volatile. Así el compilador no supone que su valor sigue igual entre lecturas. No significa que una operación sea atómica.

El patrón bandera consiste en que la ISR cambia un booleano y loop() hace el trabajo largo cuando detecta el aviso. Es una manera sencilla de separar las tareas.

Ejemplo básico:

```
volatile bool alerta = false;
void IRAM_ATTR isrBoton() { alerta = true; }
void setup() {
pinMode(21, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(21), isrBoton, FALLING);
}
void loop() {
if (alerta) { alerta = false; /* atender el botón */ }
}
```

Este ejemplo muestra la idea general. Para eventos muy rápidos o tareas simultáneas se necesitan mecanismos más cuidadosos de sincronización.

Rebote del pulsador

Al presionar un botón los contactos pueden rebotar y generar varios flancos aunque haya sido un solo toque. Por eso se utiliza antirrebote, por ejemplo ignorar cambios demasiado seguidos (unos 50 ms en el ejercicio de las diapositivas).

Con una bandera sabemos que ocurrió al menos un evento, pero no cuántas veces. Si se necesitan contar todos los pulsos, se usa un contador con la sincronización adecuada.